

FOP Mentorship Module 2 Exercise

- I. **Code Analysis:** Write the proper statement, constant(s), and variable(s) in order to arrive at the needed output.

```
/* The code below would take the value n and reverses it.
   int n = 1234957;
   int r = 0;
   while (_____)
   {
       r = _____;
       r = _____;
       n = n / 10;
   }
   printf("%d", _____);
```

```
/*The code below checks if the value n is an palindrome
/*A palindrome number is a number when reversed result to the
same number n.
/*Example: 12344321 – “is a palindrome”
          1123434211 – “is not a palindrome”

   int n = 12321;
   int original = n;
   int reversed = 0;

   while (_____)
   {
       reversed = _____;
       reversed = _____;
       n = _____;
   }

   if (_____ == _____)
       printf("%d is a palindrome", original);
   else
       printf("%d is not a palindrome", original);
```

/*An **Armstrong number** (also known as a *narcissistic number*) is a number that is equal to the **sum of its own digits each raised to the power of the number of digits**.

/*Examples:

- $153 = 1^3 + 5^3 + 3^3 = 1 + 125 + 27 = 153$
- $9474 = 9^4 + 4^4 + 7^4 + 4^4 = 6561 + 256 + 2401 + 256 = 9474$

```
int num = 153;
int original = num;
int sum = 0;
int digits = 0;
int temp = num;

// Count number of digits
while (____) {
    digits++;
    temp = ____;
}

temp = num;
while (____) {
    int digit = ____;
    sum += ____;
    temp = ____;
}

if (____) {
    printf("%d is an Armstrong number", original);
} else {
    printf("%d is not an Armstrong number", original);
}
```

/* extracts the integer log base 2 of integer x using shift
Bits example x = 20, log base 2 of 20 is 4.332 so 4 */

```
int x = 20, e = 0, t;

t_____;
while (_____)
{
    t_____;
    _____;
}

_____;
printf("Integer exponent of %d is %d", x, e);
```

II. Program Tracing: Trace the output of the code and explain briefly how it came to be.

```
n = 13;
int ctr = 0;
while(1)
{
    if(n % 2 == 0)
        n/=2;
    else
        n = 3*n + 1;
    printf("\n%d", n);
    ctr++;
    if(n == 1)
        break;
}
printf("\n%d", ctr);
```

Output:

Explanation:

```
int x = 0, temp = 0;
for (int i=25; i<30; i++)
{
    x = i;
    temp = 0;
    while (x > 0)
    {
        temp = (temp * 16) + (x % 16);
        x /= 16;
    }

    while (temp > 0)
    {
        int digit = temp % 16;
        if (digit < 10)
            printf("%d", digit);
        else
            printf("%c", 'A' + (digit - 10));
        temp /= 16;
    }
    printf(",");
}
```

Output:

Explanation:

III. Programming on Paper

Write a C program that would output the Digital Root of a number. The Digital Root is the sum of the individual digits of a number, repeating this process until we get a one-digit result.

Ex. num=3456

$$3+4+5+6=18$$

$$1+8=9$$

The Digital Root of 3456 is 9

Write a C code (go direct to the point) that would get the factorial ($n!$, e.g. $3! = 3 \times 2 \times 1 = 6$) of odd numbers inclusive from a certain range.

Example:

Range: 3 – 6

Factorial sum of odd numbers: 126 // (since $3! + 5! = 126$)